

南開大學
Nankai University

计算机学院

并行程序设计 **HIP** 课程报告

HIP Programming Course

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 6 月 12 日

项目仓库：https://github.com/Aurora1094/NKU-ParallelProgramming

目录

1 Lab1: Vector Addition **1**
1.1 Exercise: Thread Index Calculation . . . 1
1.2 Exercise 1: GPU Specifications . . . 1
1.3 Exercise 2: Implement the Kernel . . . 2
1.4 Exercise 3: Launch Configuration Calculation . . . 2
1.5 Exercise 4: Analyze Your Results . . . 3
1.6 Exercise 5: Wavefront Analysis . . . 3
1.7 Exercise 6: Occupancy Calculation . . . 4
1.8 Exercise 7: Analyze Sub-Wavefront Results . . . 4
1.9 Challenge Exercise . . . 5

2 Lab2: Matrix Transpose **6**
2.1 Exercise 1: 2D Indexing . . . 6
2.2 Exercise 2: Understand the Index Transformation . . . 7
2.3 Exercise 3: Performance Analysis . . . 7
2.4 Exercise 4: Coalescing Analysis . . . 8
2.5 Exercise 5: Tiled Transpose Analysis . . . 9
2.6 Challenge Exercises . . . 9

3 Lab3: Histogram - Atomic Operations **12**
3.1 Exercise 1: Analyze the Problem . . . 12
3.2 Exercise 2: Understand the Optimization . . . 13
3.3 Exercise 3: Record Results . . . 13
3.4 Exercise 4: Memory Efficiency . . . 14
3.5 Exercise 5: Contention Scenarios . . . 14
3.6 Challenge Exercises . . . 15

4 Lab4: Parallel Reduction **18**
4.1 Exercise 1: Understand Tree Reduction . . . 18
4.2 Exercise 2: Analyze Occupancy Results . . . 18
4.3 Exercise 3: Think About Trade-offs . . . 19
4.4 Exercise 4: Algorithm Analysis . . . 20
4.5 Challenge Exercises . . . 20

5 Lab5: Monte Carlo Integration **24**
5.1 Exercise 1: Understand the Algorithm . . . 24
5.2 Exercise 2: Manual Calculation . . . 26
5.3 Exercise 3: Scalability Analysis . . . 26
5.4 Exercise 4: Bottleneck Analysis . . . 28
5.5 Exercise 5: Precision Considerations . . . 29
5.6 Exercise 6: Optimization Analysis . . . 29
5.7 Challenge Exercises . . . 30

6 Lab6: K-Means Clustering **34**
6.1 Exercise 1: Algorithm Analysis 34
6.2 Exercise 2: Performance Analysis 35
6.3 Exercise 3: Compute Intensity 36
6.4 Exercise 4: Memory Optimization 38
6.5 Exercise 5: Convergence Analysis 39
6.6 Challenge Exercises 40

1 Lab1: Vector Addition

1.1 Exercise: Thread Index Calculation

Fill in the global index for each thread:

Original table:

`blockIdx.x`	`blockDim.x`	`threadIdx.x`	Global Index
0	256	100	_____
2	128	50	_____
5	64	0	_____

Completed table:

`blockIdx.x`	`blockDim.x`	`threadIdx.x`	Global Index
0	256	100	100
2	128	50	306
5	64	0	320

1.2 Exercise 1: GPU Specifications

Based on the output above, record your GPU's specifications:

Original table:

Property	Your GPU Value	Significance
Device Name	_____	GPU model identifier
Compute Units	_____	Number of parallel execution units
Wavefront Size	_____	Threads per warp (**32 on RDNA**, 64 on CDNA)
Max Threads per Block	_____	Upper limit for blockDim
Max Shared Memory per Block	_____	LDS available per block

Completed table:

Property	Your GPU Value	Significance
Device Name	AMD Ryzen AI 9 HX 370 w/ Radeon 890M	GPU model identifier
Compute Units	24	Number of parallel execution units
Wavefront Size	32	Threads per warp (**32 on RDNA**, 64 on CDNA)
Max Threads per Block	1024	Upper limit for blockDim
Max Shared Memory per Block	未记录	LDS available per block

Question: How many total threads can your GPU execute simultaneously if all CUs are fully utilized?

Your calculation:

$$24 \times 32 = 768 \text{ threads}$$

1.3 Exercise 2: Implement the Kernel

Complete the kernel in `vector_add_kernel.hip` based on the structure above.

Original code skeleton:

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {
    // Step 1: Calculate global thread index
    int idx = _____; // Fill in

    // Step 2: Boundary check (important when N is not divisible by block size)
    if (_____) { // Fill in the condition
        // Step 3: Perform computation
        C[idx] = _____; // Fill in
    }
}
```

Completed code:

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {
    // Step 1: Calculate global thread index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Step 2: Boundary check (important when N is not divisible by block size)
    if (idx < N) {
        // Step 3: Perform computation
        C[idx] = A[idx] + B[idx];
    }
}
```

1.4 Exercise 3: Launch Configuration Calculation

Given $N = 1000$ elements, complete the table:

Original table:

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\lceil 1000/64 \rceil$	_____	_____	_____
128	$\lceil 1000/128 \rceil$	_____	_____	_____
256	$\lceil 1000/256 \rceil$	_____	_____	_____
512	$\lceil 1000/512 \rceil$	_____	_____	_____

Completed table:

Block Size	Grid Size Formula	Grid Size	Total Threads	Idle Threads
64	$\lceil 1000/64 \rceil$	16	1024	24
128	$\lceil 1000/128 \rceil$	8	1024	24
256	$\lceil 1000/256 \rceil$	4	1024	24
512	$\lceil 1000/512 \rceil$	2	1024	24

Question: Which block size gives the best thread utilization? Why might this not always be the best choice?

Your answer:

这四种 block size 的线程利用率相同，均为：

$$\frac{1000}{1024} \times 100\% \approx 97.66\%$$

但是线程利用率最高不一定代表性能最好，因为实际性能还会受到 wavefront 大小、occupancy、寄存器使用量、内存访问效率和线程调度开销等因素影响。

1.5 Exercise 4: Analyze Your Results

Based on the multi-trial results above:

Which block size achieved the lowest mean execution time?

Block size 256，平均执行时间为 0.1411 ms。

Which block size achieved the highest memory bandwidth?

Block size 256。

Are the differences between block sizes statistically significant (look at error bars)?

差异不太显著。不同 block size 的 error bars 存在重叠，因此这些性能差异可能受到测量波动影响。

Why does Vector Add performance depend more on bandwidth than on block size?

Your answer:

Vector Add 对每个元素只进行一次浮点加法，但需要从全局内存读取 `A[i]` 和 `B[i]`，并将结果写回 `C[i]`。因此该 kernel 的计算量较小，而内存访问量相对较大，属于 memory-bound 程序。所以内存带宽对性能的影响比 block size 更明显。

1.6 Exercise 5: Wavefront Analysis

Question 1: A kernel is launched with block size 100. How many wavefronts does each block require? How many lanes are wasted?

Wavefronts per block: _____

Wasted lanes: _____

Filled result:

Wavefronts per block: 4

Wasted lanes: 28

Question 2: If the same kernel were compiled with `-mwavefrontsize64`, how would the answers change?

Wavefronts per block (wave64): _____

Wasted lanes (wave64): _____

Filled result:

Wavefronts per block (wave64): 2

Wasted lanes (wave64): 28

Question 3: What happens when threads within a wavefront take different branches (if-else)?

Your answer:

当同一个 wavefront 内的线程进入不同的 if-else 分支时，会发生 branch divergence。GPU 会将不同分支串行执行，并用 mask 屏蔽当前不属于该分支的线程，因此会降低并行效率。

1.7 Exercise 6: Occupancy Calculation

For Radeon 8060S (RDNA 3.5): 20 CUs, Warp Size = 32, Max Blocks per CU = 64, Max Threads per Block = 1024.

To calculate occupancy, you need to know the max wavefronts per CU (check rocminfo output).

Original table:

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	limited by max blocks (64)	64	_____
64	2	_____	_____	_____
128	4	_____	_____	_____
256	8	_____	_____	_____
512	16	_____	_____	_____
1024	32	_____	_____	_____

Completed table:

Block Size	Wavefronts/Block	Blocks that fit	Active Wavefronts	Notes
32	1	limited by max blocks (64)	64	limited by max blocks
64	2	32	64	limited by max wavefronts
128	4	16	64	limited by max wavefronts
256	8	8	64	limited by max wavefronts
512	16	4	64	limited by max wavefronts
1024	32	2	64	limited by max wavefronts

1.8 Exercise 7: Analyze Sub-Wavefront Results

Based on the multi-trial results above:

Mean time for block size 8 (sub-wave): _____ ms

Mean time for block size 16 (sub-wave): _____ ms

Mean time for block size 32 (wave-aligned): _____ ms

Mean time for block size 256 (large): _____ ms

Filled result:

Mean time for block size 8 (sub-wave): 0.1636 ms

Mean time for block size 16 (sub-wave): 0.1465 ms

Mean time for block size 32 (wave-aligned): 0.1496 ms

Mean time for block size 256 (large): 0.1472 ms

Question 1: Is there a noticeable performance penalty for sub-wavefront block sizes (8, 16) compared to wave-aligned sizes (32, 64, 128)?

Your answer:

部分存在。Block size 为 8 时比 wave-aligned 的 block size 更慢，但 block size 为 16 时与 wave-aligned 情况接近，甚至略快。因此本实验中 sub-wavefront 带来的性能惩罚并不稳定。

Question 2: On this RDNA 3.5 GPU, the wavefront size is 32. If you were on an MI100 (wavefront = 64), which block sizes would become sub-wavefront?

Your answer:

Block size 为 8、16 和 32 时会变成 sub-wavefront，因为它们都小于 MI100 的 wavefront size 64。

1.9 Challenge Exercise

Modify the kernel to process multiple elements per thread. This is a common technique when N is very large.

Original code skeleton:

```
__global__ void vector_add_multi(const float* A, const float* B, float* C,
                                 int N, int elements_per_thread) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = ____________________________________________; // Fill in: total number of threads

    for (int i = tid; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}
```

Completed code:

```
__global__ void vector_add_multi(const float* A, const float* B, float* C,
                                 int N, int elements_per_thread) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x; // Fill in: total number of threads

    for (int i = tid; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}
```

Question: What is the advantage of this approach over the 1:1 thread-to-element mapping?

Your answer:

这种方法使用 grid-stride loop，使每个线程可以处理多个元素。当 N 很大、元素数量超过启动线程总数时，该方法仍然可以正确处理所有数据。同时它可以复用线程，使 kernel 比严格的一线程对应一个元素的方式更加灵活。

2 Lab2: Matrix Transpose

2.1 Exercise 1: 2D Indexing

Given `blockDim = (32, 32)`, calculate the global indices:

Original table:

`blockIdx`	`threadIdx`	Global `(idx, idy)`
(0, 0)	(5, 10)	_____
(1, 0)	(0, 0)	_____
(2, 3)	(15, 20)	_____

Completed table:

`blockIdx`	`threadIdx`	Global `(idx, idy)`
(0, 0)	(5, 10)	(5, 10)
(1, 0)	(0, 0)	(32, 0)
(2, 3)	(15, 20)	(79, 116)

Question: For a matrix with `rows=100, cols=200`, how many blocks are needed?

Your calculation:

$$\texttt{blocks_x} = \left\lceil \frac{\texttt{cols}}{32} \right\rceil = \left\lceil \frac{200}{32} \right\rceil = 7$$

$$\texttt{blocks_y} = \left\lceil \frac{\texttt{rows}}{32} \right\rceil = \left\lceil \frac{100}{32} \right\rceil = 4$$

$$\texttt{blocksPerGrid} = (7, 4)$$

$$\text{total blocks} = 7 \times 4 = 28$$

2.2 Exercise 2: Understand the Index Transformation

For a 4×6 matrix ($rows = 4, cols = 6$), fill in the table:

Original table:

Thread `(idx, idy)`	Input Index	Output Index
(0, 0)	_____	_____
(1, 0)	_____	_____
(0, 1)	_____	_____
(3, 2)	_____	_____

Completed table:

Thread `(idx, idy)`	Input Index	Output Index
(0, 0)	$0 \times 6 + 0 = 0$	$0 \times 4 + 0 = 0$
(1, 0)	$0 \times 6 + 1 = 1$	$1 \times 4 + 0 = 4$
(0, 1)	$1 \times 6 + 0 = 6$	$0 \times 4 + 1 = 1$
(3, 2)	$2 \times 6 + 3 = 15$	$3 \times 4 + 2 = 14$

Question: Why is the write index formula `idx * rows + idy` instead of `idx * cols + idy`?

Your answer:

因为转置后的输出矩阵尺寸是 $cols \times rows$，也就是原来的列数变成输出矩阵的行数，原来的行数变成输出矩阵的列数。在行主序存储中，线性索引公式是：

$$\text{linear index} = \text{row} \times \text{row length} + \text{col}$$

对于输出矩阵 `output[idx][idy]`，它的行号是 `idx`，列号是 `idy`，而输出矩阵每一行的长度是 `rows`，不是 `cols`。因此写入索引应该是：

$$\texttt{idx} \times \texttt{rows} + \texttt{idy}$$

2.3 Exercise 3: Performance Analysis

Record execution times:

Original table:

Test	Dimensions	Elements	Time
1	16×16	256	_____
2	128×32	4096	_____
3	1×1024	1024	_____
4	1001×2001	2003001	_____

Completed table:

Test	Dimensions	Elements	Time
1	16×16	256	0.0077 ms
2	128×32	4096	0.0108 ms
3	1×1024	1024	0.0087 ms
4	1001×2001	2003001	0.2369 ms

Question: The large test has about 500× more elements than test 2. Is the time 500× longer? Why or why not?

Your answer:

不是。Test 4 的元素数量约为 Test 2 的：

$$\frac{2003001}{4096} \approx 489$$

倍，但是执行时间只从 0.0108 ms 增加到 0.2369 ms，约为：

$$\frac{0.2369}{0.0108} \approx 21.94$$

倍，并没有增加 500×。

原因是 GPU 可以并行处理大量元素。小规模测试中，kernel launch overhead、计时开销和固定调度开销占比较大；而大规模测试能更充分利用 GPU 并行能力，这些固定开销也被更多元素摊薄。因此执行时间不会严格按照元素数量线性增长。

2.4 Exercise 4: Coalescing Analysis

For a 1024×1024 matrix with 32×32 thread blocks (one wave32 = one row of the block):

Question 1: When reading `input[idy * cols + idx]`, are consecutive threads (in `threadIdx.x`) reading consecutive addresses?

Your answer:

是的。对于同一个 wavefront 中连续的 `threadIdx.x`，`idy` 相同而 `idx` 连续变化。因此读取地址为：

$$\texttt{idy} \times \texttt{cols} + \texttt{idx}$$

其中 `idx` 连续递增，所以读取的是连续内存地址，属于 coalesced access。

Question 2: When writing `output[idx * rows + idy]`, what is the stride between addresses written by consecutive threads?

Your answer:

连续线程的 `idx` 连续变化，而 `idy` 相同。写入地址为：

$$\texttt{idx} \times \texttt{rows} + \texttt{idy}$$

因此相邻线程写入地址之间的 stride 是：

$$\texttt{rows} = 1024$$

也就是相邻线程写入位置相隔 1024 个 float。

Question 3: If each memory transaction fetches 128 bytes (32 floats), how many transactions are needed for 32 threads (one wavefront) to write non-coalesced?

Your answer:

需要 32 次 memory transactions。因为 32 个线程写入的地址不是连续的，而是相隔 1024 个 float，每个线程基本都落在不同的 128-byte memory transaction 中，所以一个 wavefront 的 32 个线程需要约 32 次 transaction。

2.5 Exercise 5: Tiled Transpose Analysis

Question 1: Why do we add `+1` to the shared memory declaration `tile[BLOCK_SIZE][BLOCK_SIZE + 1]`?

Your answer:

添加 `+1` 是为了避免 shared memory bank conflicts。矩阵转置时会按转置方向访问 shared memory，如果 tile 的宽度正好是 `BLOCK_SIZE`，多个线程可能访问到同一个 memory bank，导致 bank conflict。将第二维加 1 后，每一行的起始地址会错开，使线程访问分布到不同 bank 中，从而减少或避免 bank conflicts。

Question 2: In the write step, why do we swap `blockIdx.x` and `blockIdx.y`?

Your answer:

因为矩阵转置会交换行和列。原矩阵中的 tile 位置是 `(blockIdx.x, blockIdx.y)`，转置后应该写到输出矩阵中对应的 `(blockIdx.y, blockIdx.x)` 位置。因此在写回输出矩阵时需要交换 `blockIdx.x` 和 `blockIdx.y`，保证每个 tile 写到转置后的正确位置。

Question 3: How much shared memory does each block use for `BLOCK_SIZE=32`?

Your calculation:

$$32 \times (32 + 1) \times 4 = 32 \times 33 \times 4 = 4224 \text{ bytes}$$

2.6 Challenge Exercises

Challenge 1: Implement Tiled Transpose

Modify `main.hip` to use the shared memory tiling approach from Section 9. Compare performance with the naive version.

Completed code:

```
#define BLOCK_SIZE 32

__global__ void transpose_tiled(const float* input, float* output,
                                int rows, int cols) {
    __shared__ float tile[BLOCK_SIZE][BLOCK_SIZE + 1];
```

```
6
7       int x = blockIdx.x * BLOCK_SIZE + threadIdx.x;
8       int y = blockIdx.y * BLOCK_SIZE + threadIdx.y;
9
10      if (x < cols && y < rows) {
11          tile[threadIdx.y][threadIdx.x] = input[y * cols + x];
12      }
13
14      __syncthreads();
15
16      x = blockIdx.y * BLOCK_SIZE + threadIdx.x;
17      y = blockIdx.x * BLOCK_SIZE + threadIdx.y;
18
19      if (x < rows && y < cols) {
20          output[y * rows + x] = tile[threadIdx.x][threadIdx.y];
21      }
22  }
```

对于大测试用例 1001×2001，naive transpose 的平均执行时间为 0.1977 ms，使用 shared memory tiling 且带 padding 的 tiled transpose 平均执行时间为 0.1964 ms。

$$\text{Speedup} = \frac{0.1977}{0.1964} \approx 1.0066$$

因此，在本次实验中 tiled transpose 相比 naive transpose 只有很小的性能提升。理论上 tiled transpose 可以通过 shared memory 让全局内存读取和写入更加接近 coalesced access，从而减少 naive transpose 中非合并写入带来的开销。但是本次测试中二者时间非常接近，说明在当前平台和测试规模下，该优化带来的收益并不明显，整体性能还可能受到 kernel 调度、缓存行为和硬件特性的影响。

Challenge 2: Bank Conflict Analysis

Test with and without the `+1` padding in shared memory. Measure the performance difference.

Code comparison:

Without padding:

```
1  __shared__ float tile[BLOCK_SIZE][BLOCK_SIZE];
```

With padding:

```
1  __shared__ float tile[BLOCK_SIZE][BLOCK_SIZE + 1];
```

对于大测试用例 1001×2001，不加 padding 的 tiled transpose 平均执行时间为 0.1980 ms，加 `+1` padding 的 tiled transpose 平均执行时间为 0.1964 ms。

$$\text{Improvement} = \frac{0.1980}{0.1964} \approx 1.0081$$

因此，加入 `+1` padding 后性能略有提升。`+1` padding 的作用是减少 shared memory bank conflicts。矩阵转置时，线程会按转置方向访问 shared memory，如果 tile 的宽度正好是 `BLOCK_SIZE`，多个线程

可能访问同一个 bank，导致访问被串行化。加入 padding 后，每一行的起始地址发生偏移，使访问更容易分布到不同 bank 中。不过本次实验中提升幅度较小，说明 bank conflict 不是唯一瓶颈。

Challenge 3: In-Place Transpose

For square matrices, implement in-place transpose using only $O(1)$ extra memory.

Completed code:

```
__global__ void transpose_inplace(float* A, int N) {
    int j = blockIdx.x * blockDim.x + threadIdx.x;
    int i = blockIdx.y * blockDim.y + threadIdx.y;

    if (i < N && j < N && i < j) {
        float temp = A[i * N + j];
        A[i * N + j] = A[j * N + i];
        A[j * N + i] = temp;
    }
}
```

对于方阵，可以只交换矩阵上三角和下三角中对称位置的元素。对于位置 (i, j)，只在 $i < j$ 时交换：

$$A[i][j] \leftrightarrow A[j][i]$$

这样每一对元素只交换一次，不需要额外输出矩阵，因此额外空间复杂度为 $O(1)$。

这种方法只适用于方阵，因为非方阵转置后矩阵尺寸会从 $rows \times cols$ 变成 $cols \times rows$，无法简单地在原数组中直接完成位置交换。

Challenge 4: Rectangular Tile Sizes

Experiment with non-square tile sizes, e.g., 32×8 or 16×64. When might these be beneficial?

Your answer:

非方形 tile 在某些情况下可能更有优势。例如 32×8 的 tile 可以保持 x 方向上 32 个连续线程访问连续内存地址，有利于 memory coalescing，同时减少每个 block 的线程数量和 shared memory 使用量。

当矩阵形状非常不规则，例如行数远大于列数或列数远大于行数时，非方形 tile 可以更好地匹配矩阵形状，减少边界处无效线程数量。此外，较小的 tile 高度可能有助于提高 occupancy，降低 shared memory 和寄存器压力。

因此，矩形 tile 可能在以下情况中有益：

- 矩阵行列比例不均衡；
- 方形 tile 导致边界无效线程较多；
- shared memory 或寄存器资源限制 occupancy；
- 希望在保持 coalesced access 的同时减少每个 block 的资源占用。

Challenge 5: Double-Buffering

Implement double-buffering to overlap memory transfers with computation for very large matrices.

Your answer:

Double-buffering 的核心思想是使用两个缓冲区交替工作：当一个 buffer 正在被 GPU kernel 处理时，另一个 buffer 同时进行下一块数据的传输。这样可以将数据传输和计算重叠起来，减少 GPU 等待数据的时间。

对于非常大的矩阵，可以将矩阵划分为多个 tile 或 chunk，然后使用两个 host/device buffer 轮流处理：

- Buffer 0 负责当前 tile 的计算；
- Buffer 1 同时传输下一 tile；
- 当前 tile 计算完成后，两个 buffer 交换角色；
- 使用 HIP stream 实现异步拷贝和 kernel 执行。

伪代码如下：

```
for each tile chunk:
    async copy next chunk to device buffer[next]
    launch transpose kernel on device buffer[current]
    async copy result of current chunk back to host
    swap current and next buffers
```

这种方法适合矩阵规模非常大、数据传输时间明显影响总运行时间的情况。通过重叠数据传输和计算，可以减少等待时间，提高整体吞吐率。

3 Lab3: Histogram - Atomic Operations

3.1 Exercise 1: Analyze the Problem

Question 1: How many times is each input element read from global memory?

Your answer:

每个 input element 会被读取 `num_bins` 次。因为 bad implementation 中每个 block 负责一个 bin，而每个 block 都会扫描整个 input array，所以同一个输入元素会被所有 bin 对应的 block 各读取一次。

Question 2: If `num_bins = 256` and $N = 10M$, what is the total global memory traffic?

Your calculation:

每个整数占 4 bytes。该方法会对输入数组重复扫描 `num_bins` 次，因此全局内存读取流量为：

$$256 \times 10,000,000 \times 4 = 10,240,000,000 \text{ bytes}$$

约为：

$$10.24 \text{ GB}$$

Question 3: Why is this approach inefficient?

Your answer:

该方法效率低的原因是每个 bin 都需要单独扫描整个输入数组，导致同一份 input data 被重复读取很多次。当 `num_bins` 很大时，全局内存访问量会变成：

$$N \times \texttt{num_bins}$$

这远大于实际需要的 N 次读取。大量重复的 global memory reads 会浪费内存带宽，并且每个 block 只负责一个 bin，整体并行方式不够高效。

3.2 Exercise 2: Understand the Optimization

Question 1: Why is atomicAdd on shared memory faster than on global memory?

Your answer:

因为 shared memory 位于每个 block 内部，访问延迟低、带宽高，atomicAdd 只在同一个 block 内部竞争。而 global memory 位于更外层，访问延迟更高，并且来自不同 block 的线程都可能同时更新同一个全局 histogram bin，竞争范围更大。因此 shared memory 上的 atomicAdd 通常比 global memory 上的 atomicAdd 更快。

Question 2: In the merge step, how many global atomicAdd operations occur per bin?

Your answer:

每个 bin 会发生 `gridDim.x` 次 global atomicAdd。因为每个 block 都会先在自己的 shared memory 中生成一个局部 histogram，然后在 merge step 中，每个 block 都会把自己的每个 bin 结果合并到 global histogram 中，所以每个 bin 对应每个 block 一次 global atomicAdd。

Question 3: What is the purpose of the grid-stride loop pattern `for (int i = idx; i < N; i += totalthread)`?

Your answer:

grid-stride loop 的作用是让所有线程共同遍历整个输入数组，即使 N 大于启动的线程总数，也能保证所有元素都被处理。每个线程从自己的全局索引 `idx` 开始，每次跳过总线程数 `totalthread`，处理多个输入元素。这样可以提高线程复用率，并且让 kernel 能适应不同大小的输入数据。

3.3 Exercise 3: Record Results

Fill in the execution times:

Original table:

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	_____	_____
Naive GPU	_____	_____
Optimized GPU	_____	_____

Completed table:

Implementation	Time (testcase 2)	Time (testcase 4)
Serial (CPU)	2.3730 ms	22.8142 ms
Naive GPU	21.2558 ms	2561.6380 ms
Optimized GPU	0.4683 ms	4.4495 ms

Calculate speedups:

- Optimized vs Serial:

$$\frac{22.8142}{4.4495} \approx 5.13\times$$

- Optimized vs Naive:

$$\frac{2561.6380}{4.4495} \approx 575.71\times$$

3.4 Exercise 4: Memory Efficiency

Question: Calculate the memory traffic reduction ratio:
Original formula:

$$\text{Reduction} = \frac{\text{Naive reads}}{\text{Optimized reads}} = \frac{_____}{_____} = _____$$

Completed formula:

$$\text{Reduction} = \frac{\text{Naive reads}}{\text{Optimized reads}} = \frac{N \times \texttt{num_bins}}{N} = \texttt{num_bins}$$

Your answer:
优化版本相比 naive 版本的 global memory read traffic 减少比例为 `num_bins` 倍。若 `num_bins` = 256，则：

$$\text{Reduction} = 256$$

也就是说，optimized approach 只需要读取每个 input element 一次，而 naive approach 会对每个 bin 都重新扫描整个输入数组，因此 naive approach 的读取量是 optimized approach 的 256 倍。

3.5 Exercise 5: Contention Scenarios

Scenario 1: All input values are the same (e.g., all zeros)
Original prompts:

- What happens to performance? _____
- Why? _____

Filled result:

- What happens to performance? 性能会下降。
- Why? 因为所有线程都会更新同一个 histogram bin，导致 atomic contention 非常严重。多个线程对同一个地址执行 `atomicAdd` 时会被串行化，无法充分并行执行，所以 kernel 时间变长。

Scenario 2: Input values are uniformly distributed across all bins
Original prompts:

- Expected contention level: _____
- Why? _____

Filled result:

- Expected contention level: 较低。
- Why? 因为输入值均匀分布在不同 bins 中，线程更新的目标地址比较分散，不会集中竞争同一个 bin。因此 atomic contention 较少，性能通常更好。

根据实验结果，Uniform 分布的平均时间为 0.4700 ms，Gaussian 分布的平均时间为 0.4677 ms，而 All-same(0) 的平均时间为 0.6851 ms。All-same 情况最慢，说明当所有线程都竞争同一个 bin 时，atomic contention 会明显降低性能。

3.6 Challenge Exercises

Challenge 1: Implement Warp-Level Histogram
Modify the kernel to use warp-level privatization:

- Each warp maintains its own local histogram
- Reduces shared memory contention further

Completed code:

```
#define BLOCK_SIZE 256
#define WARP_SIZE 32
#define WARPS_PER_BLOCK (BLOCK_SIZE / WARP_SIZE)

__global__ void histogram_warp_level(const int* input, int* histogram,
                                     int N, int num_bins) {
    __shared__ int warp_hist[WARPS_PER_BLOCK][1024];

    int tid = threadIdx.x;
    int global_tid = blockIdx.x * blockDim.x + threadIdx.x;
    int total_threads = blockDim.x * gridDim.x;
```

```
12
13      int warp_id = tid / WARP_SIZE;
14      int lane_id = tid % WARP_SIZE;
15
16      for (int bin = tid; bin < num_bins * WARPS_PER_BLOCK; bin += blockDim.x) {
17          int w = bin / num_bins;
18          int b = bin % num_bins;
19          warp_hist[w][b] = 0;
20      }
21
22      __syncthreads();
23
24      for (int i = global_tid; i < N; i += total_threads) {
25          int value = input[i];
26          atomicAdd(&warp_hist[warp_id][value], 1);
27      }
28
29      __syncthreads();
30
31      for (int bin = tid; bin < num_bins; bin += blockDim.x) {
32          int sum = 0;
33          for (int w = 0; w < WARPS_PER_BLOCK; w++) {
34              sum += warp_hist[w][bin];
35          }
36          atomicAdd(&histogram[bin], sum);
37      }
38  }
```

Warp-level histogram 的思想是让每个 warp 都有自己的一份局部 histogram，而不是整个 block 共用同一份 shared memory histogram。这样可以把原来 block 内所有线程对同一个 shared bin 的竞争，拆分成多个 warp-local bins 的竞争，从而进一步降低 shared memory atomic contention。

这种方法的优点是减少了同一个 shared memory histogram bin 上的竞争。特别是在数据分布高度 skewed 的情况下，例如大量输入值都集中在同一个 bin，warp-level privatization 可以把竞争分散到多个 warp-local histograms 中，从而提升性能。

Challenge 2: Benchmark with Different Distributions

Modify `geninput.py` to generate:

1. Uniform distribution
2. Gaussian distribution (most values near center)
3. Highly skewed (90% of values in one bin)

Compare performance for each distribution.

Completed table:

Distribution	Mean Time
Uniform	0.4700 ms
Gaussian	0.4677 ms
All-same(0)	0.6851 ms

Uniform 和 Gaussian 的执行时间接近，说明当数据分布较分散时，atomic contention 较低，性能较好。All-same(0) 的执行时间明显更高，因为所有输入值都落在同一个 bin 中，导致大量线程同时对同一个 histogram bin 执行 atomicAdd，产生最大程度的 atomic contention。

因此，数据分布越集中，atomic contention 越严重，性能越差；数据分布越均匀，更新操作越分散，性能越好。

Challenge 3: Vary Number of Bins

Test performance with `num_bins = {16, 64, 256, 1024}`.

- How does bin count affect shared memory usage?
- At what point does the optimization become less effective?

Your answer:

bin 数量会直接影响 shared memory 的使用量。优化版本中每个 block 都会在 shared memory 中维护一个局部 histogram，因此 shared memory usage 近似为：

$$\text{Shared memory per block} = \texttt{num_bins} \times 4 \text{ bytes}$$

例如：

$$16 \times 4 = 64 \text{ bytes}$$

$$64 \times 4 = 256 \text{ bytes}$$

$$256 \times 4 = 1024 \text{ bytes}$$

$$1024 \times 4 = 4096 \text{ bytes}$$

当 `num_bins` 增大时，每个 block 需要更多 shared memory，这可能降低 occupancy。同时，初始化 shared histogram 和合并 local histogram 到 global histogram 的开销也会增加。

当 bin 数量非常大，例如超过 shared memory 能高效容纳的范围，或者导致每个 block 的 shared memory 使用过高时，这种 shared memory privatization 的优化效果会变弱。此时可能需要使用 multi-pass histogram、分层归约，或者基于排序的方法来进一步优化。

4 Lab4: Parallel Reduction

4.1 Exercise 1: Understand Tree Reduction

Question 1: For an array of 1024 elements with block size 256, how many reduction steps occur within each block?

Your answer:

每个 block 有 256 个线程，因此 block 内 tree reduction 的步数为：

$$\log_2(256) = 8$$

所以每个 block 内会发生 8 个 reduction steps。

Question 2: After block-level reduction, how many partial sums remain?

Your answer:

数组共有 1024 个元素，每个 block 处理 256 个元素，因此 block 数量为：

$$\frac{1024}{256} = 4$$

每个 block reduction 后会产生一个 partial sum，所以最后剩下 4 个 partial sums。

Question 3: Why do we use `__syncthreads()` after each reduction step?

Your answer:

因为每一步 reduction 都会让部分线程把结果写入 shared memory，而下一步 reduction 会继续读取这些结果。如果没有 `__syncthreads()`，某些线程可能还没完成写入，其他线程就已经开始读取，导致读取到错误或未更新的数据。因此需要在每一步后同步 block 内所有线程，保证当前 step 的计算全部完成后，再进入下一步。

4.2 Exercise 2: Analyze Occupancy Results

From the test output, fill in the table:

Original table:

Block Size	Warps/Block	Occupancy (%)	Execution Time (ms)
64	_____	_____	_____
128	_____	_____	_____
256	_____	_____	_____
512	_____	_____	_____
1024	_____	_____	_____

Completed table:

Block Size	Warps/Block	Occupancy (%)	Execution Time (ms)
64	2	100.00	0.0849
128	4	100.00	0.0644
256	8	100.00	0.0631
512	16	100.00	0.0762
1024	32	100.00	0.0773

Question: Does higher occupancy always mean better performance?

Your answer:

不一定。从本次结果可以看到，所有 block size 的 occupancy 都是 100%，但是执行时间不同。其中 block size 为 256 时执行时间最低，为 0.0631 ms；而 block size 为 64、512、1024 时虽然 occupancy 也是 100%，但执行时间更高。

这说明 occupancy 只是影响性能的因素之一，并不能单独决定最终性能。实际性能还会受到 block size、同步次数、shared memory 访问模式、线程调度开销、memory bandwidth 和 reduction 过程中活跃线程数量变化等因素影响。

4.3 Exercise 3: Think About Trade-offs

Question 1: With block size 64, how many final atomic operations occur for $N = 1M$ elements?

Your calculation:

$$\begin{aligned}\texttt{grid_size} &= \left\lceil \frac{1,000,000}{64} \right\rceil \\ &= 15625\end{aligned}$$

因此 final atomic operations 数量为 15625。

Question 2: With block size 1024, how many final atomic operations occur?

Your calculation:

$$\begin{aligned}\texttt{grid_size} &= \left\lceil \frac{1,000,000}{1024} \right\rceil \\ &= 977\end{aligned}$$

因此 final atomic operations 数量为 977。

Question 3: Why might fewer atomic operations improve performance?

Your answer:

因为每个 block 最后都会对全局输出变量执行一次 `atomicAdd`。如果 block 数量很多，就会有更多线程块竞争同一个 global memory location，导致 atomic operations 被串行化执行，增加等待时间。减少 final atomic operations 可以降低 global atomic contention，从而减少同步和内存访问开销，提高整体性能。

4.4 Exercise 4: Algorithm Analysis

Question 1: In tree reduction, what fraction of threads are idle after the first step?

Your answer:

在 tree reduction 的第一步中，只有前一半线程继续参与计算，后一半线程变为空闲。因此空闲线程比例为：

$$\frac{1}{2} = 50\%$$

也就是说，第一步之后有一半线程处于 idle 状态。

Question 2: Why does warp shuffle not require `__syncthreads()`?

Your answer:

因为 warp shuffle 只在同一个 warp 内部进行数据交换。一个 warp 内的线程以 lockstep 方式执行，同一条指令会同步执行，因此不需要像 shared memory reduction 那样使用 `__syncthreads()` 来同步整个 block。

另外，warp shuffle 直接通过寄存器在线程之间传递数据，不需要先写入 shared memory 再读取，所以可以减少同步开销和 shared memory 访问开销。

Question 3: How does multi-element per thread improve memory bandwidth utilization?

Your answer:

Multi-element per thread 让每个线程在进入 tree reduction 之前先处理多个输入元素，并在寄存器中累加局部和。这样可以减少 block 数量和最终的 global atomicAdd 次数，同时让每个线程完成更多连续的 global memory reads，提高线程复用率。

相比每个线程只处理一个元素，这种方法减少了 kernel 中用于同步和归约的相对开销，并且能更充分地利用全局内存带宽。因此在实验结果中，Multi-Element + Tree + Shuffle 的执行时间最低，性能最好。

4.5 Challenge Exercises

Challenge 1: Implement Tree Reduction

Modify `main.hip` to use shared memory tree reduction instead of naive atomics.

Completed code:

```
__global__ void tree_reduction(const int* input, int* output, int N) {
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (idx < N) ? input[idx] : 0;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
```

```
        __syncthreads();
    }

    if (tid == 0) {
        atomicAdd(output, sdata[0]);
    }
}
```

Tree reduction 的核心思想是先在每个 block 内使用 shared memory 做局部归约，每个 block 最后只产生一个 partial sum。这样相比 naive atomic reduction，global atomicAdd 的次数从每个元素一次减少为每个 block 一次，因此可以显著降低 global atomic contention。

在本次 benchmark 中，Naive Atomic 的执行时间为 0.1177 ms，而 Tree (Shared Memory) 的执行时间为 0.0816 ms：

$$\text{Speedup} = \frac{0.1177}{0.0816} \approx 1.44\times$$

因此，shared memory tree reduction 比 naive atomic reduction 更快。

Challenge 2: Add Warp Shuffle

For the last 64 threads, use warp shuffle intrinsics:

- `__shfl_down()` for AMD GPUs
- Eliminate `__syncthreads()` for intra-warp operations

Completed code:

```
__global__ void reduction_warp_shuffle(const int* input, int* output, int N) {
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (idx < N) ? input[idx] : 0;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride >= warpSize; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid < warpSize) {
        int val = sdata[tid];

        for (int offset = warpSize / 2; offset > 0; offset >>= 1) {
            val += __shfl_down(val, offset);
        }
```

```
23
24          if (tid == 0) {
25              atomicAdd(output, val);
26          }
27      }
28  }
```

Warp shuffle 的作用是在同一个 warp 内直接通过寄存器在线程之间传递数据，不需要再通过 shared memory，也不需要使用 `__syncthreads()`。因为同一个 warp 内的线程以 lockstep 方式执行，所以 intra-warp reduction 可以更高效。

在本次 benchmark 中，Tree (Shared Memory) 的执行时间为 0.0816 ms，Tree + Warp Shuffle 的执行时间为 0.0808 ms：

$$\text{Speedup over Naive} = \frac{0.1177}{0.0808} \approx 1.46\times$$

Warp shuffle 相比普通 tree reduction 有小幅提升，说明减少 shared memory 访问和同步开销可以进一步改善性能。

Challenge 3: Multi-Pass Reduction

For very large arrays, implement a two-pass reduction:

1. First pass: Reduce to $N/$`block_size` partial sums
2. Second pass: Reduce partial sums to final result

Completed code:

```
1  __global__ void reduce_pass1(const int* input, int* partial, int N) {
2      extern __shared__ int sdata[];
3
4      int tid = threadIdx.x;
5      int idx = blockIdx.x * blockDim.x + threadIdx.x;
6
7      sdata[tid] = (idx < N) ? input[idx] : 0;
8      __syncthreads();
9
10     for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
11         if (tid < stride) {
12             sdata[tid] += sdata[tid + stride];
13         }
14         __syncthreads();
15     }
16
17     if (tid == 0) {
18         partial[blockIdx.x] = sdata[0];
19     }
20 }
```

```
__global__ void reduce_pass2(const int* partial, int* output, int num_partials) {
    extern __shared__ int sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    sdata[tid] = (idx < num_partials) ? partial[idx] : 0;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid == 0) {
        atomicAdd(output, sdata[0]);
    }
}
```

Multi-pass reduction 的思想是避免所有 block 直接对同一个 global output 做 atomicAdd。第一阶段中，每个 block 只把自己的局部结果写入一个 partial sums 数组；第二阶段再对这些 partial sums 继续做 reduction，直到得到最终结果。

这种方法适合非常大的数组。相比 single-pass 中大量 block 同时对一个 global output 执行 atomicAdd，multi-pass reduction 可以先把结果分散写入 partial sums 数组，再逐步归约，因此可以减少 global atomic contention，提高可扩展性。

Challenge 4: Benchmark Different Operations

Modify the kernel to compute:

- Maximum value
- Minimum value
- Product

Compare performance differences.

Code snippets:

Maximum reduction:

```
sdata[tid] = (idx < N) ? input[idx] : INT_MIN;

for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
    if (tid < stride) {
        sdata[tid] = max(sdata[tid], sdata[tid + stride]);
    }
    __syncthreads();
}
```

Minimum reduction:

```
sdata[tid] = (idx < N) ? input[idx] : INT_MAX;

for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
    if (tid < stride) {
        sdata[tid] = min(sdata[tid], sdata[tid + stride]);
    }
    __syncthreads();
}
```

Product reduction:

```
sdata[tid] = (idx < N) ? input[idx] : 1;

for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
    if (tid < stride) {
        sdata[tid] *= sdata[tid + stride];
    }
    __syncthreads();
}
```

从性能角度看，sum、max 和 min 的开销通常比较接近，因为它们都只需要一次简单的整数运算。Product 也属于简单算术操作，但如果数据范围较大，容易发生 overflow，因此实际使用时需要注意数值范围。

总体来说，这些 reduction 操作的性能差异通常小于算法结构带来的差异。也就是说，naive atomic、tree reduction、warp shuffle 和 multi-element per thread 这些策略对性能的影响，通常比把操作从 sum 改成 max、min 或 product 更明显。

5 Lab5: Monte Carlo Integration

5.1 Exercise 1: Understand the Algorithm

Question 1: Why is the division by `n_samples` done in each thread instead of once at the end?

Your answer: _____

Question 2: What is the time complexity of the atomic reduction approach?

Your answer: _____

Question 3: How could you modify this kernel to use shared memory reduction instead of atomics?

Your answer: _____

Answer:

Question 1: Why is the division by `n_samples` done in each thread instead of once at the end?

Your answer:

这里每个线程计算的是该 sample 对最终积分结果的贡献：

$$\texttt{tmp} = \frac{(b-a) \times y_i}{\texttt{n_samples}}$$

这样所有线程通过 `atomicAdd` 累加后，直接得到最终积分结果：

$$result \approx (b-a) \times \frac{1}{n} \sum_{i=1}^{n} y_i$$

如果不在每个线程中除以 `n_samples`，也可以先累加所有 y_i，最后再统一乘以 $(b-a)/n$。但是当前写法把每个 sample 的贡献提前计算好，使 kernel 输出的 `result` 直接就是积分近似值。

Question 2: What is the time complexity of the atomic reduction approach?

Your answer:

从总工作量来看，每个 sample 都会被一个线程处理一次，因此总计算量是：

$$O(n)$$

但是该实现中所有线程都对同一个 `result` 执行 `atomicAdd`，这些 atomic operations 会发生严重竞争，很多更新会被串行化。因此虽然线程读取和计算是并行的，但最终累加阶段会受到 atomic contention 影响，性能可能接近串行瓶颈。

Question 3: How could you modify this kernel to use shared memory reduction instead of atomics?

Your answer:

可以让每个 block 先在 shared memory 中做局部 reduction。每个线程先计算自己的 `tmp`，存入 shared memory，然后在 block 内用 tree reduction 把这些值加成一个 block-level partial sum。最后只让每个 block 的 `threadIdx.x == 0` 对全局 `result` 执行一次 `atomicAdd`。

这样可以把 global `atomicAdd` 的次数从每个 sample 一次减少到每个 block 一次，从而显著降低 global atomic contention。

示例思路如下：

```
___global___ void montecarlo_shared(const double* y_samples, double* result,
                                    double a, double b, int n_samples) {
    extern ___shared___ double sdata[];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    double tmp = 0.0;
    if (idx < n_samples) {
        tmp = (b - a) * y_samples[idx] / n_samples;
    }

    sdata[tid] = tmp;
    ___syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        ___syncthreads();
    }
```

```
22
23      if (tid == 0) {
24          atomicAdd(result, sdata[0]);
25      }
26  }
```

5.2 Exercise 2: Manual Calculation

For Test 1, verify the result manually:
Given: $a = 0$, $b = 2$, $n = 8$ samples
Formula:

$$result = \frac{(b-a)}{n} \sum_{i=1}^{n} y_i = \frac{2}{8} \times \sum y_i$$

Your calculation:

- Sum of y values: _____
- Expected result: _____
- Does it match the program output? _____

Answer:
For Test 1, the sample values are:

$$4.1805,\ 1.7864,\ 4.7554,\ 3.9983,\ 2.2786,\ 1.1873,\ 4.1222,\ 1.442$$

The sum of y values is:

$$\sum y_i = 4.1805 + 1.7864 + 4.7554 + 3.9983 + 2.2786 + 1.1873 + 4.1222 + 1.442 = 23.7507$$

Using the Monte Carlo integration formula:

$$result = \frac{2}{8} \times 23.7507 = 0.25 \times 23.7507 = 5.937675$$

- Sum of y values: 23.7507
- Expected result: 5.937675
- Does it match the program output? Yes, it matches the program output 5.937675.

5.3 Exercise 3: Scalability Analysis

Record the execution times:

Test	Sample Count	Execution Time
4	100,000	_____
7	10,000,000	_____
8	100,000,000	_____

Question 1: When sample count increases 100x (from test 4 to 7), how much does execution time increase?

Your answer: _____

Question 2: Is the scaling linear? Why or why not?

Your answer: _____

Answer:

Record the execution times:

Test	Sample Count	Execution Time
4	100,000	0.114 s
7	10,000,000	0.938 s
8	100,000,000	8.243 s

Question 1: When sample count increases 100x (from test 4 to 7), how much does execution time increase?

Your answer:

From Test 4 to Test 7, the sample count increases from $100,000$ to $10,000,000$, which is $100\times$.

The execution time increases from 0.114 s to 0.938 s:

$$\frac{0.938}{0.114} \approx 8.23$$

因此，执行时间大约增加了：

$$8.23\times$$

Question 2: Is the scaling linear? Why or why not?

Your answer:

不是完全线性。虽然 sample count 从 Test 4 到 Test 7 增加了 $100\times$，但是执行时间只增加了约 $8.23\times$。这说明在较小规模时，程序运行时间中包含较多固定开销，例如程序启动、文件读取、GPU 初始化和 kernel launch overhead，因此小规模测试不能充分体现 GPU 的并行能力。

当 sample count 增大时，GPU 有更多线程可以并行处理数据，硬件利用率更高，所以执行时间不会按样本数量完全线性增长。

从 Test 7 到 Test 8，sample count 增加 $10\times$，执行时间从 0.938 s 增加到 8.243 s：

$$\frac{8.243}{0.938} \approx 8.79$$

这个结果更接近线性 scaling，说明在大规模输入下，运行时间主要受数据处理量和内存带宽影响。

5.4 Exercise 4: Bottleneck Analysis

Question 1: If the computation takes 1 cycle and atomic takes 100 cycles, what is the theoretical efficiency for N=1M threads?

Hint: Total work = N × (compute + atomic)

Your calculation: _____

Question 2: How would shared memory reduction improve this?

Your answer: _____

Question 3: With 256 threads per block and N=1M, how many global atomics would shared memory reduction require?

Your calculation: _____

Answer:

Question 1: If the computation takes 1 cycle and atomic takes 100 cycles, what is the theoretical efficiency for N=1M threads?

Hint: Total work = N × (compute + atomic)

Your calculation:

每个线程的 compute 只需要 1 cycle，而 atomic 需要 100 cycles，因此每个线程总开销为：

$$1 + 100 = 101 \text{ cycles}$$

对于 $N = 1,000,000$ 个线程，总 work 为：

$$1,000,000 \times 101 = 101,000,000 \text{ cycles}$$

其中真正用于计算 sample contribution 的部分只有：

$$1,000,000 \times 1 = 1,000,000 \text{ cycles}$$

所以理论效率为：

$$\frac{1}{101} \times 100\% \approx 0.99\%$$

因此理论效率大约只有 0.99%。这说明性能主要被 atomic operation 限制，而不是被 sample 的独立计算限制。

Question 2: How would shared memory reduction improve this?

Your answer:

Shared memory reduction 可以让每个 block 先在 shared memory 中完成局部求和。这样 block 内的多个线程不需要全部直接对同一个 global `result` 执行 `atomicAdd`，而是先把一个 block 内的结果归约成一个 partial sum。

最后只需要每个 block 执行一次 global `atomicAdd`。这样可以把 global atomic operations 的数量从每个 sample 一次减少到每个 block 一次，大幅降低 global atomic contention。因此 shared memory reduction 可以减少串行化的 atomic 瓶颈，提高整体性能。

Question 3: With 256 threads per block and N=1M, how many global atomics would shared memory reduction require?

Your calculation:

Shared memory reduction 中，每个 block 最后只执行一次 global `atomicAdd`。因此 global atomics 数量等于 block 数量：

$$\text{num_blocks} = \left\lceil \frac{1,000,000}{256} \right\rceil$$

$$\frac{1,000,000}{256} = 3906.25$$

所以：

$$\lceil 3906.25 \rceil = 3907$$

因此 shared memory reduction 只需要 3907 次 global atomic operations，而不是 $1,000,000$ 次。

5.5 Exercise 5: Precision Considerations

Question 1: The kernel uses `double` (64-bit). How many significant digits does double precision provide?

Your answer: _____

Question 2: If we switch to `float` (32-bit), what problems might occur with N=100M samples?

Your answer: _____

Answer:

Question 1: The kernel uses `double` (64-bit). How many significant digits does double precision provide?

Your answer:

`double` 是 64-bit floating point，一般可以提供大约 15 到 16 位十进制有效数字。因此相比 `float`，`double` 在大量样本累加时可以保留更多精度，减少舍入误差。

Question 2: If we switch to `float` (32-bit), what problems might occur with N=100M samples?

Your answer:

如果改成 `float`，由于 `float` 只有大约 6 到 7 位十进制有效数字，当 $N = 100M$ 时需要累加非常多的 sample contribution，容易产生明显的 floating point rounding error。

具体来说，当当前累加和已经很大时，后续较小的数值可能因为精度不足而被舍入掉，导致部分 sample contribution 对最终结果几乎没有影响。此外，GPU 中 atomic accumulation 的执行顺序不固定，而浮点加法不满足严格结合律，因此不同运行顺序可能带来轻微不同的结果。

因此，使用 `float` 可能导致积分结果误差变大，尤其是在样本数量很大、数值范围差异较大，或者需要高精度结果时问题更明显。

5.6 Exercise 6: Optimization Analysis

Question 1: In the optimized version, how many global atomics occur for N=1M with 256 blocks?

Your answer: _____

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x)`?

Your answer: _____

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer: _____

Answer:

Question 1: In the optimized version, how many global atomics occur for N=1M with 256 blocks?

Your answer:

在 optimized version 中，每个 block 先在 shared memory 中完成局部求和，最后只由该 block 的 `threadIdx.x == 0` 执行一次 global `atomicAdd`。

因此 global atomics 的数量等于 block 数量。题目给出有 256 blocks，所以：

$$\text{global atomics} = 256$$

也就是说，相比原始版本中每个 sample 都执行一次 global atomic，该优化版本只需要 256 次 global atomic operations。

Question 2: What is the purpose of the grid-stride loop `for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x)`?

Your answer:

grid-stride loop 的作用是让每个线程可以处理多个 sample，而不是只处理一个 sample。线程从自己的全局索引 `idx` 开始，每次增加整个 grid 的线程总数：

$$\texttt{blockDim.x} \times \texttt{gridDim.x}$$

这样即使 `n_samples` 大于启动的线程总数，所有 sample 仍然都可以被覆盖和处理。

这种方式可以提高线程复用率，减少需要启动的 blocks 数量，并且让 kernel 能够适应不同规模的输入数据。

Question 3: Why is `(b - a) / n_samples` multiplication done only by thread 0?

Your answer:

因为 $\frac{b-a}{n}$ 是一个常数因子，可以在求和之后再统一乘上。原始 Monte Carlo 公式为：

$$result = \frac{b-a}{n} \sum_{i=1}^{n} y_i$$

在 optimized version 中，每个 block 先对 y_i 做局部求和，得到 `sdata[0]`。然后只让 `threadIdx.x == 0` 把该 block 的 partial sum 乘以 $\frac{b-a}{n}$，再累加到全局 result。

这样做和每个线程都先计算 $(b-a)y_i/n$ 再累加在数学上等价，但可以减少每个线程重复执行乘法和除法的开销。同时只让 thread 0 执行 global `atomicAdd`，也减少了 global atomic contention。

5.7 Challenge Exercises

Challenge 1: Implement Shared Memory Reduction

Modify `main.hip` to use the two-phase approach shown in Section 9. Compare performance.

Answer:

原始实现中，每个 sample 都会对全局变量 `result` 执行一次 global `atomicAdd`。当样本数量很大时，大量线程会竞争同一个 global memory location，导致 atomic operations 被串行化，成为性能瓶颈。

优化后的 shared memory reduction 版本先让每个 block 在 shared memory 中完成局部求和，然后每个 block 只执行一次 global `atomicAdd`。

```
__global__ void montecarlo_optimized(const double* y_samples, double* result,
                                     double a, double b, int n_samples) {
    __shared__ double sdata[256];

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    double local_sum = 0.0;

    for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x) {
        local_sum += y_samples[i];
    }

    sdata[tid] = local_sum;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] += sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid == 0) {
        atomicAdd(result, (b - a) * sdata[0] / n_samples);
    }
}
```

对于 Test 7，$N = 10,000,000$，测试结果如下：

Version	Result	Kernel Time (ms)
Naive double	9996.980601	39.874016
Optimized double	9996.980601	0.923536

加速比为：

$$\frac{39.874016}{0.923536} \approx 42.758686$$

因此，shared memory reduction 优化版本大约比 naive double 版本快：

$$42.76\times$$

该优化有效的主要原因是 global atomic operations 的数量大幅减少。Naive version 中，每个 sample 都执行一次 global `atomicAdd`；optimized version 中，每个 block 先在 shared memory 中完成局部 reduction，最后只执行一次 global `atomicAdd`。因此 global atomic contention 明显降低，性能显著提

升。

Challenge 2: Compute Pi Using Monte Carlo

Classic application: estimate π by sampling points in a square and counting how many fall inside a quarter circle.

$$\pi \approx 4 \times \frac{\text{points inside circle}}{\text{total points}}$$

Answer:

Monte Carlo 估算 π 的方法是在单位正方形 $[0,1] \times [0,1]$ 中随机生成点 (x, y)。如果满足：

$$x^2 + y^2 \leq 1$$

说明该点落在 quarter circle 内。由于 quarter circle 的面积为：

$$\frac{\pi}{4}$$

而单位正方形面积为 1，所以有：

$$\frac{\text{points inside circle}}{\text{total points}} \approx \frac{\pi}{4}$$

因此：

$$\pi \approx 4 \times \frac{\text{points inside circle}}{\text{total points}}$$

GPU kernel 可以让每个线程判断一个 sample point 是否落在 quarter circle 内：

```
__global__ void estimate_pi_kernel(const double* xs, const double* ys,
                                   int* inside_count, int n_samples) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx >= n_samples) return;

    double x = xs[idx];
    double y = ys[idx];

    if (x * x + y * y <= 1.0) {
        atomicAdd(inside_count, 1);
    }
}
```

最后在 CPU 端计算：

```
double pi_estimate = 4.0 * inside_count / n_samples;
```

这个问题适合 GPU，因为每个 sample point 的判断彼此独立，没有数据依赖，属于 embarrassingly parallel computation。主要瓶颈在最后统计 `inside_count` 的 reduction 或 atomic accumulation。

Challenge 3: Error Analysis

For a known integral, for example:

$$\int_0^1 x^2 dx = \frac{1}{3}$$

Run with $N = 1000,\ 10000,\ 100000,\ 1000000$, calculate absolute error for each, and verify the $1/\sqrt{n}$ convergence rate.

Answer:

已知真实积分值为：

$$\int_0^1 x^2 dx = \frac{1}{3} \approx 0.3333333333$$

Monte Carlo 计算得到的结果记为 I_N，则绝对误差为：

$$\text{Absolute Error} = \left| I_N - \frac{1}{3} \right|$$

实验结果如下：

Sample Count	Monte Carlo Result	Absolute Error	$\sqrt{N} \times$ Error
1,000	0.3264280000	0.0069053333	0.2183658134
10,000	0.3310900000	0.0022433333	0.2243333333
100,000	0.3330700000	0.0002633333	0.0832773117
1,000,000	0.3336350000	0.0003016667	0.3016666667

整体来看，当 N 从 $1,000$ 增加到 $100,000$ 时，absolute error 明显减小，这符合 Monte Carlo integration 的收敛规律：

$$\text{Error} \propto \frac{1}{\sqrt{N}}$$

不过误差并不是严格单调下降。例如 $N = 1,000,000$ 时的误差略大于 $N = 100,000$ 时的误差。这是 Monte Carlo 方法的正常现象，因为结果依赖随机采样。增加样本数量会降低期望误差，但不能保证每一次运行的误差都比更小样本数时更低。

$\sqrt{N} \times$ Error 列仍然处在相近数量级，因此整体上支持 $1/\sqrt{N}$ 的收敛趋势。

Challenge 4: Compare float vs double

Modify the kernel to use `float` instead of `double`. Test with large N and compare:

- Accuracy of result
- Execution time

Answer:

对于 Test 7，$N = 10,000,000$，测试结果如下：

Version	Result	Kernel Time (ms)
Optimized double	9996.980601	0.923536
Optimized float	9996.984375	0.520986

Optimized float 和 optimized double 的结果差异为：

$$|9996.984375 - 9996.980601| = 0.003774$$

Optimized float 相比 optimized double 的加速比为：

$$\frac{0.923536}{0.520986} \approx 1.77$$

因此，optimized float 版本大约比 optimized double 版本快：

$$1.77\times$$

但是 float 版本的结果与 double 版本相差约 0.003774。

这体现了 precision 和 performance 之间的 trade-off。`float` 使用 32-bit 浮点数，占用更少 memory bandwidth，通常执行速度更快；但是 `float` 只有大约 6 到 7 位十进制有效数字，而 `double` 有大约 15 到 16 位十进制有效数字。因此在大量样本累加时，`float` 更容易产生 rounding error。

所以，当更重视速度时，可以使用 `float`；当更重视数值精度时，应使用 `double`。

6 Lab6: K-Means Clustering

6.1 Exercise 1: Algorithm Analysis

Question 1: In the assignment kernel, what is the time complexity per thread if $k = 100$?

Your answer: _____

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer: _____

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer: _____

Answer:

Question 1: In the assignment kernel, what is the time complexity per thread if $k = 100$?

Your answer:

在 assignment kernel 中，每个线程负责处理一个 data point。为了找到最近的 centroid，每个线程需要遍历所有 k 个 centroid，并计算当前 point 到每个 centroid 的距离。

因此每个线程的时间复杂度是：

$$O(k)$$

当 $k = 100$ 时，每个线程需要进行 100 次 distance comparison，所以可以写成：

$$O(100)$$

由于 100 是固定常数，也可以认为对于该具体输入，每个线程执行固定数量的比较操作。但是从算法角度看，复杂度仍然是 $O(k)$。

Question 2: Why does the recalculation kernel use shared memory atomics before global atomics?

Your answer:

Recalculation kernel 需要把属于同一个 cluster 的点坐标累加起来，然后再计算 centroid 的平均值。如果所有线程都直接对 global memory 中的 centroid sum 和 count 执行 `atomicAdd`，会产生大量 global atomic contention，性能会很差。

所以该 kernel 先使用 shared memory atomics，在每个 block 内部做局部累加：

block-local sum/count

然后再把每个 block 的局部结果合并到 global memory。这样可以把大量 global atomics 转换成更快的 shared memory atomics，并减少 global memory 上的竞争。

也就是说，shared memory atomics 的作用是先做 block-level aggregation，从而降低 global atomic contention，提高 update step 的性能。

Question 3: What happens if a centroid has no points assigned to it (empty cluster)?

Your answer:

如果某个 centroid 没有任何 point 被分配给它，那么该 cluster 的 point count 为 0。这种情况下不能执行除法计算 mean，否则会出现除以 0 的问题。

在给出的 finalize kernel 中，如果：

$$\texttt{count} = 0$$

则不会重新计算该 centroid，而是保留上一轮的 centroid 坐标：

```
new_centroids_x[idx] = prev_centroids_x[idx]
```

```
new_centroids_y[idx] = prev_centroids_y[idx]
```

因此，empty cluster 的处理方式是保持原来的 centroid 不变，避免除以 0，同时保证算法可以继续运行。

6.2 Exercise 2: Performance Analysis

Record the execution time for test 4:

- Sample size: 50,000
- Clusters (k): 50
- Max iterations: 100
- Execution time: _____

Question: What is the dominant computation in each iteration?

Your answer: _____

Answer:

Record the execution time for test 4:

- Sample size: 50,000

- Clusters (k): 50
- Max iterations: 100
- Execution time: 0.105 s

Question: What is the dominant computation in each iteration?
Your answer:
每次 iteration 中最主要的计算是 assignment step，也就是为每个 data point 找到最近的 centroid。对于每个 point，kernel 需要遍历所有 k 个 centroid，并计算距离：

$$dist = (x_i - \mu_{j,x})^2 + (y_i - \mu_{j,y})^2$$

然后选择距离最小的 centroid 作为该 point 的 label。因此每个 iteration 的主要计算量大约是：

$$O(N \times k)$$

在本次 Test 4 中：

$$N = 50,000, \quad k = 50$$

所以每轮 assignment step 大约需要：

$$50,000 \times 50 = 2,500,000$$

次 distance comparison。

相比之下，recalculate centroid step 主要是对每个 point 做一次累加，并对每个 cluster 做平均，计算量大约是 $O(N + k)$。因此在 k 较大时，assignment step 通常是每轮 K-Means 中的 dominant computation。

6.3 Exercise 3: Compute Intensity

For $N = 50,000$ points and $k = 50$ clusters:
Assignment kernel:

- Distance calculations per point: _____
- Total distance calculations: _____

Update kernel:

- Shared memory atomics: _____
- Global atomics (with 256 threads/block): _____

Answer:
For $N = 50,000$ points and $k = 50$ clusters:
Assignment kernel:
每个 point 需要和所有 k 个 centroid 计算距离，因此每个 point 的 distance calculations 数量为：

$$k = 50$$

总 distance calculations 数量为：

$$N \times k = 50,000 \times 50 = 2,500,000$$

- Distance calculations per point: 50
- Total distance calculations: $2,500,000$

Update kernel:

Update phase 中，每个 point 会根据自己的 label 被累加到对应 centroid。对每个 point，需要更新：

- `new_centroids_x`
- `new_centroids_y`
- `centroid_point_nums`

因此每个 point 有 3 次 shared memory atomic operations：

$$3 \times N = 3 \times 50,000 = 150,000$$

所以 shared memory atomics 数量为：

$$150,000$$

对于 global atomics，每个 block 需要把 shared memory 中的 k 个 centroid partial sums 合并到 global memory。每个 centroid 需要更新 x、y、count 三个值，因此每个 block 的 global atomics 数量为：

$$3k = 3 \times 50 = 150$$

当 block size 为 256 时，block 数量为：

$$\left\lceil \frac{50,000}{256} \right\rceil = 196$$

所以 global atomics 总数为：

$$196 \times 150 = 29,400$$

- Shared memory atomics: $150,000$
- Global atomics (with 256 threads/block): $29,400$

6.4 Exercise 4: Memory Optimization

Question 1: The centroids are read by every thread. How could you optimize this access pattern?

Your answer: _____

Question 2: If $k = 1000$, what would be the impact on cache performance?

Your answer: _____

Answer:

Question 1: The centroids are read by every thread. How could you optimize this access pattern?

Your answer:

在 assignment kernel 中，每个线程都需要读取所有 centroid 坐标。由于同一个 block 内的线程会重复读取相同的 centroid 数据，可以把 centroid 数据先加载到 shared memory 中，然后让 block 内所有线程复用。

优化思路如下：

1. 每个 block 先把一部分或全部 `centroids_x` 和 `centroids_y` 从 global memory 加载到 shared memory。
2. 使用 `__syncthreads()` 确保 centroid 数据加载完成。
3. block 内所有线程在计算距离时从 shared memory 读取 centroid，而不是反复从 global memory 读取。

示例思路：

```
__shared__ float s_centroids_x[MAX_K];
__shared__ float s_centroids_y[MAX_K];

for (int j = threadIdx.x; j < k; j += blockDim.x) {
    s_centroids_x[j] = centroids_x[j];
    s_centroids_y[j] = centroids_y[j];
}

__syncthreads();

for (int j = 0; j < k; j++) {
    float dx = s_centroids_x[j] - data_x[point_id];
    float dy = s_centroids_y[j] - data_y[point_id];
    float dist = dx * dx + dy * dy;
}
```

这样可以减少 centroid 的 global memory 读取次数，提高数据复用率。对于 k 较小、shared memory 足够容纳所有 centroids 的情况，这种优化通常比较有效。

Question 2: If $k = 1000$, what would be the impact on cache performance?

Your answer:

如果 $k = 1000$，每个线程在 assignment step 中都需要读取 1000 个 centroid 的 x 和 y 坐标，也就是每个线程要读取 2000 个 float 值。这样 centroid 数据访问量会明显增加。

虽然 centroids 会被所有线程重复访问，理论上有较高的数据复用性，但当 k 很大时，centroid 数组可能无法很好地保留在 cache 中，导致 cache pressure 增大。如果 cache 容量不足或多个 block 同时访问大量 centroid 数据，就可能发生更多 cache miss，从而增加 global memory 访问延迟。

对于 $k = 1000$，centroid 数据大小大约为：

$$1000 \times 2 \times 4 = 8000 \text{ bytes}$$

也就是约 8 KB。这个大小本身不算特别大，但如果再考虑多个数组、多个 block、cache 竞争和更大的 k，cache performance 可能下降。

因此，当 k 较大时，可以考虑分块加载 centroids。例如每次只把一部分 centroids 加载到 shared memory 中，计算这一部分距离，再加载下一部分。这样可以减少 global memory 重复访问，并控制 shared memory 和 cache 的压力。

6.5 Exercise 5: Convergence Analysis

Question 1: What is the convergence threshold in this implementation?

Your answer: _____

Question 2: Why is it important to check convergence rather than always running `max_iterations`?

Your answer: _____

Question 3: How does the double-buffering (swapping prev and new centroids) help?

Your answer: _____

Answer:

Question 1: What is the convergence threshold in this implementation?

Your answer:

该实现中通过比较 centroid 在一轮 iteration 前后的移动距离来判断是否收敛。代码中判断条件为：

$$dx^2 + dy^2 < 1.0 \times 10^{-8}$$

因此 convergence threshold 是：

$$1.0 \times 10^{-8}$$

这里的 dx 和 dy 分别表示新旧 centroid 在 x 和 y 方向上的差值。如果某个 centroid 的平方距离变化小于 1.0×10^{-8}，就认为该 centroid 已经收敛。

Question 2: Why is it important to check convergence rather than always running `max_iterations`?

Your answer:

检查 convergence 可以避免不必要的 iteration。如果所有 centroids 已经基本不再移动，继续运行到 `max_iterations` 只会重复计算相同或几乎相同的结果，浪费 GPU 计算资源和时间。

K-Means 每一轮都需要执行 assignment step 和 update step，其中 assignment step 需要对每个 point 遍历所有 k 个 centroids，计算量较大。如果提前检测到收敛，就可以提前终止算法，从而减少 kernel launch 次数、global memory 访问次数和 atomic accumulation 开销。

因此，convergence check 可以提高程序效率，尤其是在数据较早稳定时可以明显减少总运行时间。

Question 3: How does the double-buffering (swapping prev and new centroids) help?

Your answer:

Double-buffering 使用两组 centroid 数组：一组保存上一轮的 centroids，另一组保存当前轮新计算出的 centroids。这样做可以避免在同一轮 iteration 中一边读取旧 centroid、一边覆盖写入新 centroid 所造成的数据冲突。

在 assignment step 中，所有 point 都应该基于同一组旧 centroids 来选择最近的 cluster。如果直接在原数组上更新 centroid，某些线程可能读到已经被更新过的 centroid，而其他线程还读到旧 centroid，导致同一轮计算使用的数据不一致。

使用 double-buffering 后：

`prev_centroids` → 用于 assignment 和 convergence comparison

`new_centroids` → 用于保存 update 后的新结果

每轮结束后再交换两组数组的角色。这样可以保证每一轮 iteration 的输入和输出分离，使结果更稳定，也方便比较新旧 centroid 的移动距离来判断 convergence。

6.6 Challenge Exercises

Challenge 1: Support Larger k

Modify the kernel to use dynamic shared memory allocation:

```
extern __shared__ float shared_mem[];
```

Answer:

当前实现中 shared memory 数组大小固定为 100，例如：

```
__shared__ float new_centroids_x_shared[100];
__shared__ float new_centroids_y_shared[100];
__shared__ int centroid_point_nums_shared[100];
```

这会限制最大 k 值。如果 $k > 100$，固定数组就不够用。可以改成 dynamic shared memory，在 kernel launch 时根据 k 动态分配 shared memory。

示例修改如下：

```
__global__ void recalculate_centroid_dynamic_kernel(
    const float* data_x, const float* data_y, const int* labels,
    int* centroid_point_nums,
    float* new_centroids_x, float* new_centroids_y,
    int sample_size, int k)
{
    extern __shared__ unsigned char shared_mem[];

    float* new_centroids_x_shared = (float*)shared_mem;
    float* new_centroids_y_shared =
        (float*)&new_centroids_x_shared[k];
    int* centroid_point_nums_shared =
        (int*)&new_centroids_y_shared[k];
```

```
14
15      for (int j = threadIdx.x; j < k; j += blockDim.x) {
16          new_centroids_x_shared[j] = 0.0f;
17          new_centroids_y_shared[j] = 0.0f;
18          centroid_point_nums_shared[j] = 0;
19      }
20
21      __syncthreads();
22
23      int point_id = blockIdx.x * blockDim.x + threadIdx.x;
24
25      if (point_id < sample_size) {
26          int cid = labels[point_id];
27
28          atomicAdd(&new_centroids_x_shared[cid], data_x[point_id]);
29          atomicAdd(&new_centroids_y_shared[cid], data_y[point_id]);
30          atomicAdd(&centroid_point_nums_shared[cid], 1);
31      }
32
33      __syncthreads();
34
35      for (int j = threadIdx.x; j < k; j += blockDim.x) {
36          atomicAdd(&new_centroids_x[j], new_centroids_x_shared[j]);
37          atomicAdd(&new_centroids_y[j], new_centroids_y_shared[j]);
38          atomicAdd(&centroid_point_nums[j], centroid_point_nums_shared[j]);
39      }
40  }
```

Kernel launch 时需要传入 shared memory 大小：

```
size_t shared_bytes =
    2 * k * sizeof(float) + k * sizeof(int);

recalculate_centroid_dynamic_kernel<<<grid, block, shared_bytes>>>(
    d_data_x, d_data_y, d_labels,
    d_centroid_point_nums,
    d_new_centroids_x, d_new_centroids_y,
    sample_size, k);
```

这样 shared memory 的大小会根据 k 动态变化，不再被固定的 100 限制。不过这种方法仍然受到 GPU 每个 block 最大 shared memory 容量限制。如果 k 非常大，例如 $k = 1000$ 或更高，可能还需要分块处理 centroids 或使用 hierarchical reduction。

Challenge 2: Add Distance Output

Modify the program to output the sum of squared distances (inertia) after clustering.

Answer:

K-Means 的 inertia 是所有点到其所属 centroid 的 squared distance 之和：

$$\text{inertia} = \sum_{i=0}^{N-1} \left[(x_i - \mu_{\text{label}_i,x})^2 + (y_i - \mu_{\text{label}_i,y})^2 \right]$$

可以增加一个 kernel，在 clustering 完成后计算 inertia：

```
___global___ void compute_inertia_kernel(
    const float* data_x, const float* data_y,
    const float* centroids_x, const float* centroids_y,
    const int* labels,
    float* inertia,
    int sample_size)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx >= sample_size) return;

    int cid = labels[idx];

    float dx = data_x[idx] - centroids_x[cid];
    float dy = data_y[idx] - centroids_y[cid];

    float dist = dx * dx + dy * dy;

    atomicAdd(inertia, dist);
}
```

Host 端需要分配并初始化 `d_inertia`：

```
float* d_inertia;
float h_inertia = 0.0f;

hipMalloc(&d_inertia, sizeof(float));
hipMemset(d_inertia, 0, sizeof(float));

compute_inertia_kernel<<<grid, block>>>(
    d_data_x, d_data_y,
    d_centroids_x, d_centroids_y,
    d_labels,
    d_inertia,
    sample_size);

hipMemcpy(&h_inertia, d_inertia, sizeof(float), hipMemcpyDeviceToHost);
```

最后输出 inertia：

```
std::cout << "Inertia: " << h_inertia << std::endl;
```

Inertia 可以用来衡量 clustering 的紧密程度。值越小，说明点离自己的 centroid 越近，聚类结果通常越紧凑。

Challenge 3: Extend to 3D

Add support for 3D point clustering by adding z coordinates.

Answer:

要把 2D K-Means 扩展到 3D，需要增加一组点坐标 `data_z` 和一组 centroid 坐标 `centroids_z`。距离公式也要从 2D squared distance 改成 3D squared distance：

$$dist = (x_i - \mu_{j,x})^2 + (y_i - \mu_{j,y})^2 + (z_i - \mu_{j,z})^2$$

Assignment kernel 可以修改为：

```
__global__ void find_nearest_centroids_3d_kernel(
    const float* data_x, const float* data_y, const float* data_z,
    int* labels,
    const float* centroids_x,
    const float* centroids_y,
    const float* centroids_z,
    int sample_size, int k)
{
    int point_id = blockIdx.x * blockDim.x + threadIdx.x;

    if (point_id >= sample_size) return;

    int nearest = 0;
    float min_dist = INFINITY;

    for (int j = 0; j < k; j++) {
        float dx = centroids_x[j] - data_x[point_id];
        float dy = centroids_y[j] - data_y[point_id];
        float dz = centroids_z[j] - data_z[point_id];

        float dist = dx * dx + dy * dy + dz * dz;

        if (dist < min_dist) {
            min_dist = dist;
            nearest = j;
        }
    }

    labels[point_id] = nearest;
}
```

Update kernel 也需要多累加一个 z 坐标：

```
atomicAdd(&new_centroids_x_shared[cid], data_x[point_id]);
atomicAdd(&new_centroids_y_shared[cid], data_y[point_id]);
atomicAdd(&new_centroids_z_shared[cid], data_z[point_id]);
atomicAdd(&centroid_point_nums_shared[cid], 1);
```

Finalize kernel 中也需要对 z 坐标求平均：

```
new_centroids_x[idx] /= count;
new_centroids_y[idx] /= count;
new_centroids_z[idx] /= count;
```

Convergence check 也要加入 z 方向：

```
float dx = new_centroids_x[idx] - prev_centroids_x[idx];
float dy = new_centroids_y[idx] - prev_centroids_y[idx];
float dz = new_centroids_z[idx] - prev_centroids_z[idx];

if (dx * dx + dy * dy + dz * dz < 1.0e-8f) {
    atomicAdd(num_converged_centroids, 1);
}
```

因此，3D 扩展的核心变化是增加 `z` 坐标数组，并把 assignment、update 和 convergence check 中的距离计算从二维扩展到三维。

Challenge 4: K-Means++ Initialization

Implement K-Means++ initialization on GPU:

1. Choose first centroid randomly
2. For each subsequent centroid, choose with probability proportional to distance squared

Answer:

K-Means++ 的目的是选择更好的初始 centroids，减少普通随机初始化导致的局部最优问题。它的基本思想是：新的 centroid 更倾向于选择离已有 centroids 较远的点。

步骤如下：

1. 随机选择第一个 centroid。
2. 对每个 point，计算它到最近已选 centroid 的 squared distance，记为 D_i^2。
3. 按照概率 $\frac{D_i^2}{\sum D_i^2}$ 选择下一个 centroid。
4. 重复直到选出 k 个 centroids。

GPU 上可以把距离计算并行化：

```
__global__ void compute_min_distances_kernel(
    const float* data_x, const float* data_y,
    const float* centroids_x, const float* centroids_y,
    float* min_distances,
    int sample_size, int num_selected)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx >= sample_size) return;

    float best = INFINITY;

```

```
    for (int j = 0; j < num_selected; j++) {
        float dx = data_x[idx] - centroids_x[j];
        float dy = data_y[idx] - centroids_y[j];
        float dist = dx * dx + dy * dy;

        if (dist < best) {
            best = dist;
        }
    }

    min_distances[idx] = best;
}
```

然后需要对 `min_distances` 做 parallel reduction 或 prefix sum，得到总和并根据随机数选择下一个 centroid。

K-Means++ 的优点是通常能得到更好的初始 centroids，使 K-Means 更快收敛，并且降低收敛到较差 local optimum 的概率。缺点是 initialization 本身比随机初始化更复杂，需要多轮距离计算和采样，因此会增加初始化开销。

Challenge 5: Mini-Batch K-Means

Process random subsets of points each iteration to speed up convergence on very large datasets.

Answer:

Mini-Batch K-Means 每轮 iteration 不使用全部 N 个 data points，而是随机选取一小部分样本作为 mini-batch 来更新 centroids。

普通 K-Means 每轮 assignment 的复杂度大约为：

$$O(N \times k)$$

Mini-Batch K-Means 如果每轮只使用 B 个样本，其中 $B \ll N$，则每轮复杂度变为：

$$O(B \times k)$$

因此，对于非常大的 dataset，Mini-Batch K-Means 可以显著减少每轮计算量。

GPU 实现思路如下：

1. 每轮随机生成或读取一个 mini-batch index array。
2. assignment kernel 只处理这些 mini-batch points。
3. update kernel 只根据 mini-batch 中的 points 更新 centroid。
4. 多轮迭代后逐步逼近完整 K-Means 的结果。

示例 kernel 结构如下：

```
__global__ void find_nearest_centroids_minibatch_kernel(
    const float* data_x, const float* data_y,
    const int* batch_indices,
    int* labels,
    const float* centroids_x, const float* centroids_y,
```

```
    int batch_size, int k)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid >= batch_size) return;

    int point_id = batch_indices[tid];

    int nearest = 0;
    float min_dist = INFINITY;

    for (int j = 0; j < k; j++) {
        float dx = centroids_x[j] - data_x[point_id];
        float dy = centroids_y[j] - data_y[point_id];
        float dist = dx * dx + dy * dy;

        if (dist < min_dist) {
            min_dist = dist;
            nearest = j;
        }
    }

    labels[point_id] = nearest;
}
```

Mini-Batch K-Means 的优点是单轮计算更快，适合超大规模数据集。缺点是每轮只使用部分样本，因此 centroid update 会有更多随机噪声，最终结果可能没有 full-batch K-Means 稳定。实际应用中需要在 batch size、迭代次数和精度之间做 trade-off。